Coder's Gyan

Custom Protocol Document

Simple CHAT Protocol 1.0

By Coder's Gyan

1. Abstract

The Simple CHAT Protocol is a lightweight, text-based protocol designed for educational purposes. It enables a TCP-based chat system where clients can authenticate, join a chat session, send messages, and leave. This document outlines the protocol's syntax, commands, and operational flow.

2. Introduction

The CHAT protocol is intended for use in environments where simplicity and clarity are key. It serves as an educational tool to illustrate how higher-level protocols can be built on top of TCP. The protocol defines four primary commands:

- **AUTH**: Authenticate a client.
- **JOIN**: Allow an authenticated client to join the chat.
- **SEND**: Send a chat message.
- **LEAVE**: Remove a client from the chat.

3. Message Format

Each message in the CHAT protocol is composed of three parts:

1. **Start Line**: Contains the protocol identifier, version, and command.
2. **Headers**: One or more key-value pairs, including at least the `Content-Length` header.
3. **Body**: The message payload (optional, depending on the command).

A blank line (`\r\n`) separates the headers from the body.

3.1. General Structure

```
CHAT/1.0 <COMMAND>
Header1: Value1
Header2: Value2
Content-Length: <number>

<body>
```

3.2. Example Messages

```
CHAT/1.0 AUTH
User: alice
Token: secret123
Content-Length: 0
```

```
CHAT/1.0 JOIN
Content-Length: 0
```

```
CHAT/1.0 SEND
Content-Length: 28

Hello, this is Alice speaking!
```

```
CHAT/1.0 LEAVE
Content-Length: 0
```

- Success Response:

- Failure Response (if not authenticated):

4. Command Definitions

- Purpose: Authenticate the client.
- Required Headers: User, Token
- Description:The client sends credentials to the server. If the token matches the expected value (e.g., "secret123"), the server marks the connection as authenticated.
- Purpose: Join the chat session.
- Required Headers: User
- Description:An authenticated client sends this command to join the chat. The server then adds the client to the active session list and may notify other clients.
- Purpose: Send a chat message.
- Required Headers: Content-Length
- Description:After joining, the client sends a message using the `SEND` command. The server broadcasts the message to all connected clients.
- Purpose: Leave the chat session.
- Description:The client indicates that it is leaving the chat. The server removes the client from the active session list and optionally notifies the remaining clients.

5. Protocol Flow

1. The client establishes a TCP connection to the server.
2. The server may send a welcome message (outside the protocol specification) upon connection.
3. The client sends an AUTH command with User and Token headers.
4. The server validates the credentials:

- Success: Responds with CHAT/1.0 OK (with an empty body).
- Failure: Responds with an error message and terminates the connection.

1. An authenticated client sends a JOIN command.
2. The server marks the client as "joined" and may broadcast a notification to other clients.
3. The client sends a `SEND` command containing the message body.
4. The server validates that the client is both authenticated and joined.
5. The server broadcasts the message to all connected clients (excluding the sender) using the `MESSAGE` command format (similar to SEND).

- Server Response:

1. The client sends a `LEAVE` command.
2. The server removes the client from the session and broadcasts a departure message if applicable.
3. The TCP connection is then closed gracefully.

6. Error Handling

If a command is received from a client that is not authenticated or joined (where required), the server responds with an error message in the following format:

```
CHAT/1.0 ERROR
Error: <Description>
Content-Length: 0
```

Examples:

- Not authenticated: "Error: Not authenticated"
- Unknown command: "Error: Unknown command"

7. Security Considerations

- The protocol transmits credentials in plain text for educational purposes. In a production environment, additional measures (e.g., encryption or secure tokens) should be implemented.

- Token validation is simplified (e.g., token must equal "secret123").

8. Extensibility

- Future versions may include additional commands (e.g., private messaging, room management) or enhanced error codes.
- The version number in the protocol identifier (CHAT/1.0) allows for backward compatibility as changes are introduced.

← Previous: Network Fundamentals Mock Test

Next: Build our own Protocol →

0 comments so far

6

Add a comment...

Comment cannot be empty

Comment

No comments yet. Be the first to start the discussion!